Deep Technical Review — sam3d.md: Segment Platform Research Foundation Document

Claude Opus (code review)

March 2026

Contents

1 Deep Technical Review: Segment Platform Research Foundation Document

1.1 1. Overall Assessment

1.1.1 1.1 Summary Judgement

This is a remarkably thorough and self-aware research planning document. It covers clinical context, model architecture, prompting strategies, system design, and a phased roadmap across two clinical domains. The document's greatest strength is its intellectual honesty: critical caveats are flagged inline (the ACDC-vs-coronary Dice distinction, the `dense_prompt` blocker, the 128-cubed patch mismatch, the HU calibration problem). These are not buried — they are highlighted with warning boxes. This is rare and commendable in a research planning document.

However, the document conflates aspirational design with implementable specification in several places, and several core workflows depend on unresolved blockers that are acknowledged but insufficiently prioritised. The result is a document that reads as 60% solid technical reference and 40% speculative architecture.

1.1.2 1.2 Strengths

Strength	Detail
Verified model claims	Paper table/section citations are specific (Table 3, Table 5, Section 5.1.5). Not hand-waving.
Honest caveats	The ACDC cardiac Dice vs. coronary CTA distinction is correctly identified as the single most important gap.
Clinical grounding	DISCHARGE and SCOT-HEART references are real trials with correct PMID-level citations. AHA-segment nomenclature, CAD-RADS 2.0 stenosis definition, and PI-RADS v2.1 are correctly used.
Dual-domain design	Prostate mpMRI as a second domain forces generalisability into the architecture from day one.
Working code exists	The codebase contains functional MEDIS parsers, CPR generation, STL mesh construction, and plaque mask creation. This is not vapourware.

Strength	Detail
Bishop frame choice	Correctly identifies Frenet-Serret instability and chooses parallel transport. The implementation in `medis_to_cpr.py` matches the pseudocode.

1.1.3 1.3 Weaknesses

Weakness	Severity	Detail
`dense_prompt` blocker unresolved	BLOCKING	Appears in 4+ workflows. No implementation path chosen. No estimated effort.
128-cubed patch vs. coronary span	BLOCKING	Acknowledged but sliding-window stitching is described in one sentence with no implementation detail.
Normalisation gap	HIGH	SA-Med3D-140K training normalisation for CT is "not fully documented." Guessing wrong will silently degrade all results.
SCOT-HEART access not confirmed	HIGH	External validation on 4,146 patients is a headline claim but no data access agreement is mentioned.
Active learning loop is a sketch	HIGH	One paragraph. No selection strategy, no retraining frequency analysis, no catastrophic forgetting mitigation.
Thread safety of model loading	MEDIUM	Global model object shared across FastAPI thread pool workers with no locking.

Weakness	Severity	Detail
Roadmap timeline is compressed	MEDIUM	36 weeks to publication with clinical validation, two domains, and a model fine-tuning loop.
No error budget / fallback plan	MEDIUM	What if zero-shot Dice on coronary lumen is 0.40? No decision tree for when to abandon SAM-Med3D in favour of nnU-Net.

1.2 2. Section-by-Section Review

1.2.1 2.1 Abstract & Executive Summary

Assessment: Strong.

The abstract correctly scopes the document as "internal technical reference and research planning" rather than a deployment package. The "Why Segmentation Matters Here" section provides clear clinical justification.

Issues:

1. The success target of ">0.85 Dice (coronary lumen)" is stated as "ambitious but grounded." The grounding is based on nnU-Net task-specific results (0.75–0.88). But SAM-Med3D is a zero-shot foundation model, not a task-specific model. The relevant comparison is zero-shot performance on unseen anatomy, where SAM-Med3D-turbo reports much lower Dice on unseen targets. The 0.85 target should be explicitly identified as a **post-fine-tuning** target, not a zero-shot expectation.
2. The "10× cheaper than manual" cost claim has no calculation backing it. At what hourly rate? Including GPU amortisation? Including radiologist QC time?

Suggested change: Split the Dice target into two rows: "Zero-shot Dice (coronary lumen): > 0.60" and "Post-fine-tuning Dice (coronary lumen): > 0.85". This sets honest expectations and prevents premature discouragement.

1.2.2 2.2 Foundation Model: SAM-Med3D-turbo

Assessment: Very strong. The most carefully verified section of the document.

Issues:

1. **Parameter count discrepancy.** The document says “86M encoder + 5M decoder” in a note but uses “86M + 1M + 4M = 91M” in the table. These do not agree. The prompt encoder (1M) is suspiciously low for learned 3D embeddings. Recommend verifying via `sum(p.numel() for p in model.parameters())` on the actual checkpoint.
2. **Turbo variant provenance.** The turbo checkpoint is referenced from a GitHub issue comment (#2), not a published paper. The 44 additional datasets are not enumerated. This means:
    - The training data distribution is unknown.
    - There is no guarantee that the turbo checkpoint has the same license terms as the base model.
    - Reproducibility is compromised.
3. **MedIM loader dependency.** The document uses `medim.create_model("SAM-Med3D", ...)` but MedIM is a third-party wrapper library. If MedIM changes its API or is abandoned, the entire inference pipeline breaks. Pin the version explicitly in `requirements.txt`.

Suggested change: Add a “Model Provenance Risk” subsection noting the turbo checkpoint’s informal release channel and recommending the team maintain a local fork of the checkpoint and loader.

1.2.3 2.3 Clinical Domain A: Coronary CCTA (DISCHARGE)

Assessment: Strong clinical knowledge, weak implementation path.

1.2.3.1 2.3.1 Segmentation Classes The HU ranges are clinically correct for standard 120 kVp soft-kernel CT. The document correctly warns about multi-centre calibration.

Missing: No mention of motion-correction preprocessing. DISCHARGE is a multi-centre trial with mixed prospective/retrospective ECG gating. The document acknowledges motion artefacts but does not describe how gating quality metadata will be extracted from DICOM headers and used for case stratification.

1.2.3.2 2.3.2 The `dense_prompt` Blocker (Critical) This is the single most consequential technical gap in the document. It appears in:

- Section on Dual-Wall Nested Segmentation
- Prostate Lesion Segmentation with PZ prior
- Technical Challenges section
- Workflow 1, Step 5 (“Expand to Wall”)
- Phase 2 roadmap item

The document describes two paths: (a) custom prompt encoder modification, or (b) sample lumen surface into points. Neither is analysed for feasibility:

Path	Effort	Risk
(a) Modify prompt encoder	Requires understanding the 3D ViT architecture internals, modifying the forward pass, and verifying gradient flow for fine-tuning. Estimated 2–4 weeks of focused ML engineering.	May break pre-trained weights. No guarantee that a dense prior improves results — the model was never trained with dense prompts.
(b) Surface point sampling	Sample N points from the lumen mask surface as additional positive prompts. Simpler. No architecture change.	Point budget: SAM-Med3D was trained with 1–N points. What is the maximum N before the prompt encoder saturates? If lumen surface has 5000 points and you sample 50, is that sufficient spatial coverage?

Missing analysis: - What is the maximum number of prompt points SAM-Med3D can accept? The paper shows results for 1, 3, 5, 10 points. Surface sampling would require hundreds. - Has anyone in the SAM-Med3D community implemented dense prompts? The CVPR25 MedSegFM challenge (linked in the document) may have relevant solutions. - Fallback: If dense prompts fail entirely, can the EEM be obtained via morphological dilation of the lumen mask (e.g., `binary_dilation(lumen, iterations=3)`)? This is crude but may be sufficient for initial plaque burden estimates.

1.2.3.3 2.3.3 HU Plaque Characterisation

The `characterise_plaque()` function is straightforward but has a subtle maintenance risk:

```
total = vessel_wall_mask.sum()
```

The `high_risk` threshold `(lipid_rich / total) > 0.04` compares a raw voxel count ratio to 0.04, while the return value for other fields multiplies by 100 to produce a percentage. The variable names say `_pct` but the `high_risk` calculation uses the raw ratio. This is internally consistent but a maintenance hazard — a future developer adding `lipid_rich_pct` to the `high_risk` check would introduce a 100× error.

The HU calibration problem is correctly identified but under-specified. The document says “needs per-centre calibration” but does not propose a method. The aortic root normalisation approach should be proposed as the default method: measure mean HU in the ascending aorta root (always present in every CCTA and with known expected HU for a given kVp) and use it as an internal standard to correct plaque thresholds across DISCHARGE sites.

1.2.3.4 2.3.4 Stenosis Calculation

The diameter stenosis formula is correct per CAD-RADS 2.0. However, the document does not describe how `D_min` and `D_ref` will be computed from the segmentation mask. This requires: 1. Centerline extraction from the lumen mask 2. Cross-sectional area measurement at each centerline point 3. Equivalent diameter: `D = 2 * sqrt(A / pi)` 4. Reference segment selection algorithm

None of these steps are specified. The comparison with MEDIS QAngio CT requires matching the exact reference segment selection method, which the document notes but does not implement.

1.2.4 2.4 Clinical Domain B: Prostate mpMRI

Assessment: Adequate but secondary.

Issues:

1. **Multi-sequence fusion.** "Run separate inferences per sequence; fuse masks post-hoc" is stated but not designed. What fusion rule? Majority vote? Union? Intersection? The answer depends on the clinical question (whole gland vs. lesion detection).
2. **ADC map as input.** SAM-Med3D was trained on 3D volumes. An ADC map is a derived map with intensity range 0–3000 × 10□□ mm²/s, completely different from T2W signal intensity. Z-score normalisation per volume is proposed, which is reasonable, but the model has no way to know it is looking at diffusion data vs. anatomical data. This fundamentally limits zero-shot lesion detection performance.
3. **PI-RADS integration.** The document mentions mapping lesion volume to "PI-RADS size criterion" but PI-RADS scoring is multiparametric by definition — it requires T2W + DWI + DCE interpretation. A segmentation model cannot produce a PI-RADS score; it can only provide volumetric inputs to a radiologist who scores manually. This should be stated explicitly to avoid scope creep.

1.2.5 2.5 Technical Challenges

Assessment: Well-identified, weakly resolved.

1.2.5.1 2.5.1 128-Cubed Patch Size vs. Coronary Span

This is the second most critical technical gap after `dense_prompt`.

The document states: "Coronary arteries span > 128 voxels (300–400 voxels at 0.5 mm)." The proposed solution is "Sliding window with 50% overlap; Gaussian-weighted blending."

Missing details:

1. **How many patches?** A 512×512×300 CTA volume at 128-cubed patches with 50% overlap requires approximately `(512/64) * (512/64) * (300/64) ≈ 8 * 8 * 5 = 320 patches`. At 0.5–1.5 s per patch, that is 160–480 seconds — far exceeding the 2 s latency target.
2. **Selective patching.** The 2 s target is only achievable if patches are placed selectively along the vessel, not as a full sliding window. This requires a coarse vessel detection step first (e.g., thresholding HU > 200, connected component analysis, then placing patches only along candidate vessels). This pre-processing step is not described.
3. **Boundary artefacts.** Gaussian-weighted blending prevents double-counting but does not prevent the model from producing inconsistent predictions across patches. A vessel segment that is 50% in patch A and 50% in patch B may be segmented differently by each. Connected-component analysis post-stitching is mentioned but is insufficient — it can detect but not repair inconsistencies.
4. **Embedding caching scope.** The document mentions Redis embedding cache. Is the embedding computed per-patch or per-volume? If per-patch, changing the prompt requires re-running the decoder on all relevant patches. The cache stores patch-level embeddings, which is correct but needs explicit documentation.

1.2.5.2 2.5.2 Motion Artefact Handling

The `motion_robust_segmentation()` pseudocode proposes running inference on 10 cardiac phases and taking a median. The document correctly warns this is topologically unsafe and recommends optimal phase selection instead. Good self-correction.

However, `volume_4d` implies 4D data is available. DISCHARGE CCTA is typically reconstructed as a single best-phase 3D volume. Multi-phase data is only available for retrospective gating cases. The document should clarify whether DISCHARGE has multi-phase data or whether this code path is theoretical.

1.2.6 2.6 System Architecture

Assessment: Reasonable for a planning document. Several implementation risks.

1.2.6.1 2.6.1 API Design — Thread Safety (Critical)

The segmentation endpoint uses a global `model` object:

```
model = medim.create_model(...)
model.eval()

@router.post("/api/segment/point")
def segment_point(req: SegmentRequest):
```

```
    with torch.no_grad():
        mask = model.segment(volume, points=voxel_points, labels=effective_labels)
```

FastAPI runs `def` (sync) endpoints in a thread pool (`anyio.to_thread`). If two requests arrive simultaneously, both threads will call `model.segment()` concurrently on the same PyTorch model. PyTorch models are not thread-safe by default. GPU memory for intermediate activations will be allocated concurrently, causing OOM or CUDA errors.

Fix options: - Use `threading.Lock()` around the inference call. - Run Uvicorn with `--workers 1 --limit-concurrency 1`. - Use Celery with a dedicated GPU worker process.

1.2.6.2 2.6.2 Volume Loading and Path Traversal

Every request calls `nib.load(f"/data/volumes/{r`. If `volume_id` contains `../../etc/passwd`, this is a path traversal vulnerability. Input validation (alphanumeric + hyphen only) is required before any production deployment.

Additionally, there is no volume cache — every request reads 500 MB from disk. An LRU cache of recently loaded volumes would dramatically reduce I/O overhead for repeated prompts on the same volume.

1.2.6.3 2.6.3 Authentication

LDAP/Charité SSO is mentioned in the architecture but no middleware or dependency injection is shown in the endpoint code. The `segment_point()` function has no authentication check. Every endpoint must declare an auth dependency before any production deployment.

1.2.6.4 2.6.4 DICOM Conversion

The document lists `dcm2niix-wasm` for in-browser DICOM conversion. Converting a 3,561-patient CCTA study in the browser is impractical for batch mode. This should be a backend preprocessing step for batch processing, with browser-side conversion only for ad-hoc single-case loading.

1.2.7 2.7 MEDIS TXT Parser (TypeScript)

Assessment: Functional, with a latent type safety gap.

The TypeScript parser correctly resets both `current` and `points` on each `# Contour index:` line, which fixes stale value inheritance between contour blocks. This is explicitly documented.

Latent issue: If a contour block has a missing `# group:` line (malformed MEDIS file), `current.group` will be `undefined`, and `as MedisContour` silently produces an object with `group: undefined`. TypeScript type assertions do not perform runtime validation.

The Python parsers in the codebase guard against this explicitly by checking `group in contours` before storing. The TypeScript parser should add:

```
if (current.group && current.sliceDistance !== undefined && points.length > 0) {
    contours.push({ ...current, points } as MedisContour);
}
```

1.2.8 2.8 CPR / Bishop Frame Implementation

Assessment: Correct. Implementation matches specification.

The Bishop frame propagation in the document:

`N[i] = normalize(N[i-1] - dot(N[i-1], T[i]) * T[i])`

matches the actual implementation in `medis_to_cpr.py` (line 177):

```
Ni = Ni_prev - np.dot(Ni_prev, Ti) * Ti
```

The degenerate case (`norm < 1e-10`) falls back to the previous normal, which is reasonable for near-straight segments.

One concern: The initial normal selection uses a least-aligned-axis heuristic, which is standard but means the initial orientation of CPR cross-sections varies between patients. For cross-patient comparability of CPR views, the initial normal should be anatomically standardised (e.g., always pointing towards the anterior chest wall).

Performance concern: The `world_to_cpr()` transform in `transform_to_cpr.py` uses a brute-force O(N×M) nearest-centerline-point search. For a mesh with 50,000 vertices and a 500-point centerline, this is 25 million distance calculations in a Python loop. A `scipy.spatial.cKDTree` would reduce this to O(N log M).

1.2.9 2.9 Mesh Generation

Assessment: Adequate. Three-strategy classification is appropriate.

The codebase contains working mesh generation code. The `create_tube_mesh_simple()` function handles variable point counts via `resample_polygon()` and applies rotational alignment. End-capping logic (fan triangulation from centroid) is correct.

Dead code risk in `create_tube_mesh()`: The non-simple version uses vertex indexing `i * max(n1, n2) + j` but builds `all_points` by appending contours with their actual (potentially different) sizes. When `n1 ≠ n2`, vertex indices will be wrong. This function is never called (`process_file()` uses the `_simple` variant), but its presence creates a maintenance hazard. Remove it.

1.2.10 2.10 Deployment, Performance & Security

Assessment: Realistic hardware requirements. Security section needs significant work.

The memory budget is well-calculated. The 268 MB per CTA volume ($512^3 \times 2$ bytes for int16) is correct. The 4 GB VRAM estimate for FP16 SAM-Med3D-turbo is consistent with model size.

Missing: - No TLS/HTTPS configuration or certificate management. - No audit logging specification (required by GDPR to trace who accessed which patient data and when). - No data retention policy. - No model versioning strategy — when the model is fine-tuned weekly (active learning), how are model versions tracked and rolled back?

1.2.11 2.11 Research Roadmap

Assessment: Ambitious. Timeline is compressed but not impossible if scope is managed.

Phase	Weeks	Realistic?	Key Risk
1 (MVP)	1–4	Tight but feasible	Python code exists; TypeScript port is the main work
2 (SAM integration)	5–8	Insufficient for `dense_prompt`	`dense_prompt` alone is a research problem requiring 3–4 weeks
3 (DISCHARGE eval)	9–12	Feasible if data pipeline ready	DICOM→NIfTI at scale is non-trivial
4 (Fine-tuning)	13–20	Tight	nnU-Net training alone requires 5-fold CV; 8 weeks covers both fine-tuning and baseline
5 (Prostate)	21–28	Aggressive	Entire new clinical domain while maintaining coronary
6 (Validation + publication)	29–36	Unrealistic	Reader study requires IRB, recruitment, statistical power analysis

The Q1 2026 milestone ("Zero-shot baseline on DISCHARGE + MVP deployed") ends 2026-03-31 — 13 days from the document's own date. If neither deliverable is substantially complete, the roadmap is already behind.

SCOT-HEART external validation: The document lists SCOT-HEART (4,146 patients) as the external validation dataset but does not mention a data sharing agreement. SCOT-HEART

CCTA was acquired 2010–2014 with older scanner technology. Protocol differences from DISCHARGE (2015–2019) may dominate over model generalisation performance. If SCOT-HEART access is not secured, name an alternative (e.g., CAT-CAD, CONFIRM registry, or a public coronary CTA dataset).

1.2.12 2.12 Active Learning Loop

Assessment: Severely under-specified.

The document provides one paragraph. Missing design decisions:

1. **Sample selection strategy.** Which cases are shown to experts for correction? Random? Lowest model confidence? Highest disagreement between model and heuristic QC? The choice dramatically affects learning efficiency.
2. **Retraining scope.** Full fine-tuning of all 91 M parameters weekly? LoRA/adapter-only? Decoder-only? Full fine-tuning on a growing dataset becomes computationally expensive at scale.
3. **Catastrophic forgetting.** Fine-tuning on coronary CTA corrections may degrade general medical segmentation ability (prostate domain). Elastic weight consolidation (EWC) or continual learning strategies should be evaluated.
4. **Validation during retraining.** How is a new model validated before replacing the old one? Champion-challenger deployment requires a held-out test set.
5. **Correction UI.** The document mentions "local brush edits in the browser" but the frontend architecture does not include a drawing/editing tool. Niivue 0.66 has drawing support (confirmed by `draw2D`/`draw3D` demo files in the codebase), but integrating this with a mask correction export pipeline requires substantial engineering.

1.3 3. Cross-Cutting Concerns

1.3.1 3.1 The Normalisation Gap

The document states: "The exact normalisation used in SAM-Med3D's SA-Med3D-140K training pipeline is not fully documented for CT modalities."

This is a critical risk. If the model was trained with z-score normalisation per volume rather than min-max clipping to [−100, 700], the proposed `normalise_ccta()` function will produce inputs the model has never seen, silently degrading all results.

Recommended action — run before any evaluation work:

```
normalisation_configs = [
    {"name": "raw",     "clip": None,           "norm": None},
```

```
    {"name": "full_ct", "clip": (-1024, 3071),  "norm": "minmax"},
    {"name": "cardio",  "clip": (-100, 700),    "norm": "minmax"},
    {"name": "zscore",  "clip": None,           "norm": "zscore"},
    {"name": "lumen",   "clip": (0, 400),       "norm": "minmax"},
]
# For each config, run SAM-Med3D on 10 held-out DISCHARGE cases
# with identical prompt points. Measure Dice.
# Select the config with highest mean Dice. Hard-code as default.
```

This experiment takes 1–2 days and prevents months of wasted effort.

1.3.2 3.2 Evaluation Metrics

The document mentions Dice and Hausdorff distance. For coronary arteries, these are necessary but insufficient:

Metric	Why Needed
Dice coefficient	Standard volumetric overlap
95th-percentile Hausdorff distance	Captures worst-case boundary error
Diameter stenosis % correlation vs. MEDIS	The actual clinical endpoint
Centreline overlap (OV, OF, OT)	Standard for tubular structure evaluation
Clinically relevant lesion detection rate	Did the model find the >50% stenosis?
Average symmetric surface distance (ASSD)	More robust than Hausdorff for thin structures

1.3.3 3.3 Comparison with nnU-Net Baseline

The roadmap mentions benchmarking against nnU-Net but provides no detail. The nnU-Net training protocol (number of annotated cases, 5-fold CV, 3d_fullres configuration, held-out test cases) must be specified upfront before the comparison is meaningful.

1.3.4 3.4 Codebase vs. Document Alignment

Document Claim	Code Status
MEDIS TXT parser	4 Python implementations (each slightly different — no canonical parser)
CPR with Bishop frame	`medis_to_cpr.py` — full implementation
STL mesh generation	`medis_to_stl.py` — full implementation

Document Claim	Code Status
Plaque mask creation	`medismask.py` — two methods (polygon, radial) □
CPR coordinate transform	`transform_to_cpr.py` — full implementation □
FastAPI backend	**No code exists — entirely aspirational**
TypeScript frontend	**No code exists — entirely aspirational**
SAM-Med3D integration	**No model loading or inference code**

1.4 4. Specific Technical Recommendations

1.4.1 4.1 Selective Patch Placement for Coronary Inference

Instead of full sliding window (320 patches, 160–480 s), use a two-stage approach:

1. **Coarse vessel detection:** Threshold HU > 200, extract connected components, filter by elongation ratio > 5:1.
2. **Targeted patch placement:** Place 128-cubed patches centered on detected vessel voxels, with 50% overlap along the vessel axis only.
3. **Estimated patch count:** 10–30 patches per vessel.
4. **Estimated latency:** 5–15 s — above the 2 s target but manageable with embedding caching.

1.4.2 4.2 Thread-Safe Inference

```
import threading
_model_lock = threading.Lock()

@router.post("/api/segment/point")
def segment_point(req: SegmentRequest):
    # ... validation and preprocessing ...
    with _model_lock:
        with torch.no_grad():
            mask = model.segment(volume, points=voxel_points, labels=effective_labe
    # ... postprocessing ...
```

Or use a dedicated inference worker process with a Celery task queue.

1.4.3 4.3 Volume ID Validation

```
import re
VOLUME_ID_PATTERN = re.compile(r'^[a-zA-Z0-9_\-]+$')
```

```python
@router.post("/api/segment/point")
def segment_point(req: SegmentRequest):
    if not VOLUME_ID_PATTERN.match(req.volume_id):
        raise HTTPException(status_code=400, detail="Invalid volume_id format.")
```

1.4.4 4.4 Aortic Root HU Calibration

For cross-site plaque threshold calibration, measure the mean HU in the ascending aorta root (always present in every CCTA, known expected HU for each kVp). Use this as an internal standard to shift the plaque HU boundaries per-case:

```python
def calibrate_hu_thresholds(aorta_roi_hu: np.ndarray, target_aorta_hu: float = 350.
    measured_mean = aorta_roi_hu.mean()
    offset = target_aorta_hu - measured_mean
    return {
        "calcified_threshold": 130 + offset,
        "fibrous_lower": 60 + offset,
        "fibrous_upper": 130 + offset,
        "lipid_threshold": 60 + offset,
    }
```

1.5 5. Assessment of Specific Claims

Claim	Verdict	Notes
“SAM-Med3D-turbo, 91M params”	Plausible	Verify with `sum(p.numel() ...)` on actual checkpoint
“87.12% Dice on cardiac structures”	Correct	ACDC dataset, Table 5. Not coronary CTA.
“Sub-second inference”	Correct for single 128³ patch	Multi-patch coronary: 5–30 s
“~4 GB VRAM (FP16)”	Plausible for weights	Add activation memory for actual usage
“>0.85 Dice coronary lumen”	Optimistic for zero-shot	Realistic after task-specific fine-tuning
“>0.90 Dice prostate whole-gland”	Reasonable	Large, well-defined structure
“<2 s end-to-end latency”	Unlikely for coronary	Single-patch structures only; coronary needs multi-patch

Claim	Verdict	Notes
“50 cases/hour batch throughput”	Plausible	With 4× A100s and optimised pipeline
“10× cheaper than manual”	Unsubstantiated	No cost model provided
“SCOT-HEART external validation”	Unconfirmed	No data access agreement mentioned
“Active learning: weekly re-training”	Aspirational	No implementation design

---

1.6 6. Execution List

Tasks are ordered by severity (BLOCKING > HIGH > MEDIUM > LOW) and within each level by recommended execution sequence.

1.6.1 BLOCKING — Must resolve before core pipeline is functional

#	Task	Rationale	Effort
B1	**Run normalisation sweep experiment.** Test 5 normalisation strategies on 10 held-out DISCHARGE cases with identical prompt points. Select the winner. Hard-code as the default `normalise_ccta()` parameters.	Wrong normalisation silently destroys all downstream results. 1–2 days of work that prevents months of wasted effort.	1–2 days

#	Task	Rationale	Effort
B2	**Implement selective patch placement for coronary inference.** Design coarse vessel detection + targeted patch placement. Benchmark latency and Dice vs. full sliding window on 10 cases.	Full sliding window (320 patches, 160–480 s) exceeds the 2 s latency target by 100×. Without targeted patching, coronary segmentation is impractical.	2 weeks
B3	**Resolve `dense_prompt` implementation path.** Prototype both approaches (a) encoder modification, (b) surface point sampling on 5 test cases. Measure Dice for outer wall. Select approach. Define fallback (morphological dilation) if both fail.	4+ workflows depend on this. Cannot evaluate dual-wall segmentation or plaque burden without it.	3–4 weeks

1.6.2 HIGH — Must resolve before DISCHARGE evaluation is credible

#	Task	Rationale	Effort
H1	**Confirm SCOT-HEART data access.** Initiate data sharing agreement. If not feasible within 8 weeks, document an alternative external validation dataset.	External validation is a headline claim. Cannot publish without it.	Ongoing (administrative)
H2	**Implement diameter stenosis calculation pipeline.** Centreline extraction from lumen mask, cross-sectional area, equivalent diameter, reference segment selection matching MEDIS QAngio CT method.	Clinical endpoint. Dice alone does not demonstrate clinical utility.	2–3 weeks
H3	**Add thread safety and input validation to inference endpoint.** Model lock, volume ID validation, LRU cache for loaded volumes.	Current pseudocode crashes under concurrent load. Path traversal vulnerability in volume ID.	1 week

#	Task	Rationale	Effort
H4	**Design the active learning loop.** Specify sample selection strategy, retraining scope, validation protocol, model versioning, catastrophic forgetting mitigation. Write as separate design document.	"Weekly re-training" without a design is a recipe for regression.	1–2 weeks (design), 4+ weeks (implementation)
H5	**Split Dice targets into zero-shot and post-fine-tuning.** Add: "Zero-shot Dice (coronary lumen): > 0.60" and "Post-fine-tuning: > 0.85".	Prevents discouragement when zero-shot results are inevitably lower than the 0.85 target.	30 minutes
H6	**Implement aortic root HU normalisation for cross-site plaque calibration.** Measure ascending aorta mean HU as internal standard. Calibrate per-case plaque thresholds accordingly.	The calibration problem is raised but no solution proposed. Required for valid cross-site plaque burden comparisons.	1 week

1.6.3 MEDIUM — Should resolve before Phase 4

#	Task	Rationale	Effort
M1	**Consolidate MEDIS parsers into a single canonical Python module.** Codebase has 4 different implementations. Create `medis_utils.py` imported by all downstream scripts.	Reduces maintenance burden. Current copies are already diverging.	2–3 days
M2	**Add runtime validation to TypeScript MEDIS parser.** Guard against missing `group` or `sliceDistance`. Add `Number.isFinite()` on parsed coordinates.	Prevents silent data corruption on malformed input.	1 day
M3	**Define nnU-Net baseline training protocol.** Specify: number of annotated cases, 5-fold CV, nnU-Net 3d_fullres configuration, held-out test cases.	Without a protocol, the nnU-Net comparison deliverable is undefined.	1 day

#	Task	Rationale	Effort
M4	**Add evaluation metrics beyond Dice.** Implement 95th-percentile Hausdorff, ASSD, centreline overlap (OV/OF/OT), stenosis % correlation.	Dice alone is insufficient for thin tubular structures.	1 week
M5	**Verify SAM-Med3D parameter count.** Print `sum(p.numel() for p in model.parameters())` on the turbo checkpoint. Update the document.	Document has conflicting parameter counts (91M vs. 86M+5M).	15 minutes
M6	**Remove hardcoded Windows paths from Python scripts.** Replace with command-line arguments or config file.	Prevents accidental commits of machine-specific paths. Enables CI/CD.	1 hour
M7	**Accelerate `world_to_cpr()` with KD-tree.** Replace brute-force O(N×M) search with `scipy.spatial.cKDTree`.	200× speedup for typical mesh sizes.	1–2 hours

#	Task	Rationale	Effort
M8	**Add model versioning infrastructure.** Track checkpoints with: training data version, validation Dice, deployment date. Use MLflow or DVC.	Essential for active learning loop reproducibility.	1–2 weeks
M9	**Document multi-sequence prostate MRI fusion rule.** Specify fusion strategy for T2W + ADC inference results (majority vote, union, intersection).	“Fuse masks post-hoc” is not a specification.	1 day

1.6.4 LOW — Long-term improvements

#	Task	Rationale	Effort
L1	**Add cost model for “10× cheaper” claim.** Calculate GPU amortisation, radiologist QC time, and software maintenance per case. Compare against manual segmentation.	Unsupported cost claims weaken the document.	1 day

#	Task	Rationale	Effort
L2	**Investigate maximum SAM-Med3D prompt point count.** Run inference with 10, 50, 100, 500 points. Measure Dice and latency. Determines feasibility of surface sampling for dense_prompt path (b).	Informs B3 prototype design.	1 day
L3	**Split document into multiple files.** Separate architecture, coronary protocol, prostate protocol, evaluation plan, and deployment spec. Add changelog header.	1,200+ line document will become unmanageable.	2–3 hours
L4	**Standardise CPR initial normal orientation anatomically.** Use anterior chest wall vector instead of arbitrary perpendicular for cross-patient CPR comparability.	Improves reproducibility across patients.	1 day

#	Task	Rationale	Effort
L5	**Add TLS configuration and audit logging to deployment spec.** Both are GDPR requirements for clinical deployment.	Regulatory necessity.	1–2 weeks
L6	**Investigate SCOT-HEART scanner vintage.** Determine scanner models and acquisition years. Quantify expected domain shift from DISCHARGE.	Prevents surprise failures during external validation.	1 day
L7	**Remove dead code `create_tube_mesh()` from `medis_to_stl.py`.** Function has incorrect vertex indexing for unequal contour sizes and is never called.	Code hygiene. Prevents future misuse.	1 hour
L8	**Add a decision tree for model fallback.** Define: if zero-shot Dice < X on N cases, switch to nnU-Net. If fine-tuned Dice < Y after Z corrections, escalate.	Prevents sunk-cost fallacy.	1 day

1.7 7. Conclusion

The Segment Platform Research Foundation Document is a strong piece of research planning that demonstrates deep domain knowledge across clinical cardiology, radiology, 3D medical image segmentation, and WebGL rendering. Its defining characteristic — honest inline identification of risks and caveats — is unusual and valuable.

The primary concern is the gap between the document's ambition and the current implementation state. The codebase contains mature MEDIS processing tools (CPR generation, plaque masking, mesh export), but nothing related to SAM-Med3D inference, FastAPI, or the TypeScript frontend exists yet. The three blocking items (normalisation, patch placement, dense_prompt) must be resolved before any meaningful evaluation can begin.

Recommended execution sequence: 1. **B1: Normalisation sweep** (1–2 days) — immediate, no dependencies, highest information value per unit effort. 2. **B2: Selective patch placement** (2 weeks) — enables coronary inference at all. 3. **B3: `dense_prompt` prototype** (3–4 weeks) — enables plaque burden assessment and dual-wall pipeline.

Completing B1–B3 in sequence (total ~6–7 weeks) provides the minimum viable foundation for the Phase 3 DISCHARGE evaluation milestone and the core contribution of the project.